

Advanced T-SQL Concepts (AdventureWorks2022)

This note uses the **AdventureWorks2022** database, a sample SQL Server database modeling a bicycle manufacturing and sales company. Examples leverage standard schemas like **Sales**, **Production**, **HumanResources**, and **Person**.

1. Programming Transact-SQL

Overview

Transact-SQL (T-SQL) is SQL Server's procedural extension to SQL, enabling programming constructs like variables, control-of-flow, and cursors to build complex logic in AdventureWorks2022.

Expanded Explanations and Definitions

- **Variables:** Named storage for data, declared with **DECLARE** and assigned with **SET** or **SELECT**.
 - **Sub-explanation:** Variables hold temporary values for calculations or logic.
 - **Code Example:**

```
DECLARE @TotalDue MONEY;  
SELECT @TotalDue = TotalDue FROM Sales.SalesOrderHeader WHERE  
SalesOrderID = 43659;  
SELECT @TotalDue AS OrderTotal;
```

- **Comment:** Stores and displays the total for order 43659, fulfilling variable's role as temporary storage.

- **Control-of-Flow:** Constructs like **IF**, **WHILE**, and **CASE** for conditional or iterative logic.
 - **Sub-explanation:** Directs program execution based on conditions.
 - **Code Example:**

```
DECLARE @OrderCount INT;  
SELECT @OrderCount = COUNT(*) FROM Sales.SalesOrderHeader WHERE  
CustomerID = 29500;  
IF @OrderCount > 0  
    SELECT 'Customer has orders' AS Status;  
ELSE  
    SELECT 'No orders found' AS Status;
```

- **Comment:** Checks if a customer has orders, showing conditional logic execution.

- **Cursors:** Iterate over query results row-by-row, useful for sequential processing.
 - **Sub-explanation:** Processes each row individually, though often less efficient than set-based operations.
 - **Code Example:**

```

DECLARE @ProductID INT, @Name NVARCHAR(50);
DECLARE product_cursor CURSOR FOR
    SELECT ProductID, Name FROM Production.Product;
OPEN product_cursor;
FETCH NEXT FROM product_cursor INTO @ProductID, @Name;
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT CAST(@ProductID AS NVARCHAR(10)) + ': ' + @Name;
    FETCH NEXT FROM product_cursor INTO @ProductID, @Name;
END;
CLOSE product_cursor;
DEALLOCATE product_cursor;

```

- **Comment:** Prints product IDs and names row-by-row, fulfilling cursor's iterative processing role.

Examples

Basic: Variable and IF Statement

```

DECLARE @CustomerID INT = 29500;
DECLARE @OrderCount INT;
SELECT @OrderCount = COUNT(*) FROM Sales.SalesOrderHeader WHERE CustomerID =
@CustomerID;
IF @OrderCount > 0
    SELECT 'Customer ' + CAST(@CustomerID AS NVARCHAR(10)) + ' has ' +
CAST(@OrderCount AS NVARCHAR(10)) + ' orders.' AS Result;
ELSE
    SELECT 'No orders found for Customer ' + CAST(@CustomerID AS NVARCHAR(10)) +
'.' AS Result;

```

Explanation: Counts orders for a customer and outputs a message based on the count.

Intermediate: WHILE Loop for Inventory Update

```

DECLARE @ProductID INT, @Quantity INT;
DECLARE inventory_cursor CURSOR FOR
    SELECT ProductID, Quantity FROM Production.ProductInventory WHERE Quantity <
100;
OPEN inventory_cursor;
FETCH NEXT FROM inventory_cursor INTO @ProductID, @Quantity;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE Production.ProductInventory
    SET Quantity = Quantity + 50
    WHERE ProductID = @ProductID;
    FETCH NEXT FROM inventory_cursor INTO @ProductID, @Quantity;
END;

```

```
CLOSE inventory_cursor;  
DEALLOCATE inventory_cursor;
```

Explanation: Increases inventory for products with low stock using a cursor and loop.

Advanced: Dynamic SQL with CASE

```
DECLARE @TableName NVARCHAR(50) = 'Sales.SalesOrderHeader';  
DECLARE @ColumnName NVARCHAR(50) = 'SalesPersonID';  
DECLARE @SQL NVARCHAR(500);  
DECLARE @Result NVARCHAR(100);  
SET @SQL = N'SELECT @Result = CASE WHEN COUNT(DISTINCT ' + QUOTENAME(@ColumnName)  
+ ') > 0 THEN ''Data exists'' ELSE ''No data'' END FROM ' + QUOTENAME(@TableName);  
EXECUTE sp_executesql @SQL, N'@Result NVARCHAR(100) OUTPUT', @Result OUTPUT;  
SELECT @Result AS QueryResult;
```

Explanation: Dynamically checks if a column has data, using **CASE** for conditional output.

Real-Life Use Case

Batch Processing: AdventureWorks uses T-SQL scripts to process monthly sales data, using variables to store thresholds, **WHILE** loops to iterate over orders, and dynamic SQL to generate reports across different territories, automating financial reporting.

2. Transactions

Overview

Transactions in SQL Server ensure data integrity in AdventureWorks2022 by grouping operations into atomic units that either all succeed or all fail, maintaining consistency.

Expanded Explanations and Definitions

- **Transaction:** A sequence of operations treated as a single unit, adhering to ACID properties (Atomicity, Consistency, Isolation, Durability).
 - **Sub-explanation:** Ensures all changes are committed or rolled back together.
 - **Code Example:**

```
BEGIN TRANSACTION;  
UPDATE Sales.SalesOrderHeader SET TotalDue = TotalDue * 0.9 WHERE  
SalesOrderID = 43659;  
COMMIT TRANSACTION;
```

- **Comment:** Applies a discount and commits, fulfilling the transaction's all-or-nothing property.

- **COMMIT:** Saves all changes in a transaction permanently.
 - **Sub-explanation:** Finalizes the transaction's operations.
 - **Code Example:**

```
BEGIN TRANSACTION;  
INSERT INTO Sales.SalesOrderHeader (CustomerID, OrderDate, SubTotal)  
VALUES (29500, GETDATE(), 1000);  
COMMIT;
```

- **Comment:** Inserts an order and saves it, showing commit's role in persistence.

- **ROLLBACK:** Undoes all changes if an error occurs.
 - **Sub-explanation:** Restores the database to its pre-transaction state.
 - **Code Example:**

```
BEGIN TRANSACTION;  
UPDATE Production.ProductInventory SET Quantity = Quantity - 10 WHERE  
ProductID = 1;  
IF @@ERROR <> 0  
    ROLLBACK;  
ELSE  
    COMMIT;
```

- **Comment:** Reduces inventory but rolls back on error, fulfilling rollback's recovery role.

Examples

Basic: Simple Transaction

```
BEGIN TRANSACTION;  
UPDATE Sales.SalesOrderHeader  
SET SubTotal = SubTotal + 100  
WHERE SalesOrderID = 43659;  
COMMIT;
```

Explanation: Increases an order's subtotal and commits the change.

Intermediate: Multi-Table Transaction

```
BEGIN TRANSACTION;  
UPDATE Sales.SalesOrderDetail  
SET OrderQty = OrderQty + 1  
WHERE SalesOrderID = 43659 AND ProductID = 776;  
UPDATE Production.ProductInventory  
SET Quantity = Quantity - 1
```

```
WHERE ProductID = 776 AND LocationID = 6;
IF @@ROWCOUNT = 0
    ROLLBACK;
ELSE
    COMMIT;
```

Explanation: Updates order quantity and inventory, rolling back if no rows are affected.

Advanced: Nested Transaction with Savepoint

```
BEGIN TRANSACTION OuterTran;
    INSERT INTO Sales.SalesOrderHeader (CustomerID, OrderDate, SubTotal) VALUES
(29500, GETDATE(), 5000);
    SAVE TRANSACTION OrderSaved;
    BEGIN TRANSACTION InnerTran;
        UPDATE Production.ProductInventory SET Quantity = Quantity - 50 WHERE
ProductID = 707;
        IF (SELECT Quantity FROM Production.ProductInventory WHERE ProductID =
707) < 0
            BEGIN
                ROLLBACK TRANSACTION OrderSaved;
                COMMIT; -- Commits nothing due to rollback to savepoint
            END;
        ELSE
            COMMIT; -- Commits InnerTran
    COMMIT; -- Commits OuterTran
```

Explanation: Inserts an order and updates inventory, using a savepoint to rollback only the inventory update if it causes negative stock.

Real-Life Use Case

Order Fulfillment: AdventureWorks uses transactions to process orders, updating `Sales.SalesOrderHeader`, `Sales.SalesOrderDetail`, and `Production.ProductInventory` atomically, ensuring stock and order data remain consistent even during system failures.

3. Error Handling

Overview

Error handling in T-SQL manages runtime errors in AdventureWorks2022 using `TRY...CATCH` blocks, ensuring robust scripts that gracefully handle issues like constraint violations or invalid data.

Expanded Explanations and Definitions

- **TRY...CATCH:** A construct to handle errors, executing code in `TRY` and catching errors in `CATCH`.
 - **Sub-explanation:** Captures error details for logging or recovery.
 - **Code Example:**

```
BEGIN TRY
    UPDATE Sales.SalesOrderHeader SET TotalDue = -100 WHERE
SalesOrderID = 43659;
END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE() AS Error;
END CATCH;
```

- **Comment:** Attempts an invalid update and returns the error message, fulfilling error capture.
- **THROW:** Raises a custom error or re-throws a caught error.
 - **Sub-explanation:** Allows custom error messages for specific conditions.
 - **Code Example:**

```
IF NOT EXISTS (SELECT 1 FROM Sales.SalesOrderHeader WHERE SalesOrderID
= 99999)
    THROW 50001, 'Order does not exist.', 1;
```

- **Comment:** Raises a custom error for a missing order, fulfilling custom error handling.
- **ERROR Functions:** Functions like `ERROR_NUMBER()`, `ERROR_MESSAGE()`, and `ERROR_LINE()` provide error details.
 - **Sub-explanation:** Extract error context in `CATCH` blocks.
 - **Code Example:**

```
BEGIN TRY
    INSERT INTO Sales.SalesOrderHeader (CustomerID, OrderDate) VALUES
(0, GETDATE());
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrNum, ERROR_MESSAGE() AS ErrMsg,
ERROR_LINE() AS ErrLine;
END CATCH;
```

- **Comment:** Captures error details for an invalid customer ID, fulfilling error detail extraction.

Examples

Basic: Simple TRY...CATCH

```
BEGIN TRY
    UPDATE Sales.SalesOrderDetail
    SET UnitPrice = -10
    WHERE SalesOrderID = 43659;
```

```

END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;

```

Explanation: Attempts an invalid negative price update and returns the error message.

Intermediate: Custom Error with THROW

```

BEGIN TRY
    DECLARE @CustomerID INT = 99999;
    IF NOT EXISTS (SELECT 1 FROM Person.Person WHERE BusinessEntityID =
@CustomerID)
        THROW 50001, 'Invalid Customer ID.', 1;
    INSERT INTO Sales.SalesOrderHeader (CustomerID, OrderDate, SubTotal) VALUES
(@CustomerID, GETDATE(), 1000);
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber, ERROR_MESSAGE() AS ErrorMessage;
END CATCH;

```

Explanation: Validates a customer ID before inserting an order, throwing a custom error if invalid.

Advanced: Error Logging

```

BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE Sales.SalesOrderDetail
    SET OrderQty = OrderQty + 10
    WHERE SalesOrderID = 43659 AND ProductID = 776;
    UPDATE Production.ProductInventory
    SET Quantity = Quantity - 10
    WHERE ProductID = 776 AND LocationID = 6;
    IF (SELECT Quantity FROM Production.ProductInventory WHERE ProductID = 776 AND
LocationID = 6) < 0
        THROW 50002, 'Insufficient inventory.', 1;
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK;
    INSERT INTO Production.ErrorLog (ErrorTime, UserName, ErrorNumber,
ErrorMessage)
    VALUES (GETDATE(), SUSER_SNAME(), ERROR_NUMBER(), ERROR_MESSAGE());
    SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;

```

Explanation: Updates order and inventory in a transaction, logs errors to `Production.ErrorLog`, and rolls back if inventory is insufficient.

Real-Life Use Case

Data Validation: AdventureWorks uses `TRY...CATCH` to validate order updates, logging constraint violations (e.g., negative quantities) to `Production.ErrorLog` for auditing and debugging, ensuring reliable data processing.

4. PolyBase and Query Store

Overview

PolyBase enables querying external data sources (e.g., Hadoop, Azure Blob Storage) as tables in AdventureWorks2022, while **Query Store** tracks query performance and execution plans in SQL Server, aiding optimization.

Expanded Explanations and Definitions

- **PolyBase:** A SQL Server feature to query external data using T-SQL, integrating with sources like Azure Data Lake or Hadoop.
 - **Sub-explanation:** Treats external data as SQL tables via external tables.
 - **Code Example:**

```
CREATE EXTERNAL DATA SOURCE AzureBlobStorage
WITH (
    TYPE = BLOB_STORAGE,
    LOCATION = 'https://mystorageaccount.blob.core.windows.net',
    CREDENTIAL = StorageCredential
);
```

- **Comment:** Defines a connection to Azure Blob Storage, fulfilling PolyBase's external data access role.

- **External Table:** A SQL Server table mapping to external data, queried like a local table.
 - **Sub-explanation:** Enables seamless T-SQL queries on external data.
 - **Code Example:**

```
CREATE EXTERNAL TABLE Sales.ExternalSalesData (
    SalesOrderID INT,
    TotalDue MONEY
)
WITH (
    LOCATION = 'salesdata/sales.csv',
    DATA_SOURCE = AzureBlobStorage,
    FILE_FORMAT = CsvFileFormat
);
```


- **Comment:** Creates a table for querying CSV data in Azure, fulfilling external table integration.
- **Query Store:** A SQL Server feature that captures query execution plans and performance metrics.
 - **Sub-explanation:** Helps diagnose performance issues by tracking plan changes.
 - **Code Example:**

```
ALTER DATABASE AdventureWorks2022 SET QUERY_STORE = ON;  
SELECT q.query_id, t.query_sql_text  
FROM sys.query_store_query q  
JOIN sys.query_store_query_text t ON q.query_text_id = t.query_text_id;
```

- **Comment:** Enables Query Store and lists queries, fulfilling performance tracking.

Examples

Basic: Query External Table with PolyBase

```
-- Assumes external data source and table are set up  
SELECT SalesOrderID, TotalDue  
FROM Sales.ExternalSalesData  
WHERE TotalDue > 1000;
```

Explanation: Queries sales data from an external CSV file in Azure Blob Storage.

Intermediate: Join External and Local Data

```
-- Assumes external table Sales.ExternalSalesData exists  
SELECT soh.SalesOrderID, soh.TotalDue AS LocalTotal, esd.TotalDue AS ExternalTotal  
FROM Sales.SalesOrderHeader soh  
INNER JOIN Sales.ExternalSalesData esd ON soh.SalesOrderID = esd.SalesOrderID  
WHERE soh.OrderDate >= '2014-01-01';
```

Explanation: Joins local `Sales.SalesOrderHeader` with external sales data to compare totals.

Advanced: Query Store Performance Analysis

```
SELECT  
    q.query_id,  
    t.query_sql_text,  
    p.avg_duration,  
    p.last_execution_time  
FROM sys.query_store_query q  
JOIN sys.query_store_query_text t ON q.query_text_id = t.query_text_id  
JOIN sys.query_store_plan p ON q.query_id = p.query_id
```

```
WHERE t.query_sql_text LIKE '%Sales.SalesOrderHeader%'
AND p.avg_duration > 100000; -- Queries with average duration > 100ms
```

Explanation: Identifies slow queries on `Sales.SalesOrderHeader` using Query Store, aiding optimization.

Real-Life Use Case

Data Integration and Optimization: AdventureWorks uses PolyBase to query sales data in Azure Data Lake, joining it with local `Sales.SalesOrderHeader` for consolidated reporting. Query Store tracks query performance, identifying slow joins for index tuning, improving dashboard responsiveness.

Summary

- **Programming Transact-SQL:** Uses variables, loops, and cursors for complex logic in AdventureWorks2022.
- **Transactions:** Ensures data consistency with atomic operations.
- **Error Handling:** Manages errors robustly with `TRY...CATCH` and logging.
- **PolyBase and Query Store:** Integrates external data and optimizes query performance.

Additional Resources

- SQL Server T-SQL documentation: <https://docs.microsoft.com/sql/t-sql>
- AdventureWorks2022: <https://github.com/Microsoft/sql-server-samples>
- PolyBase: <https://docs.microsoft.com/sql/relational-databases/polybase>
- Query Store: <https://docs.microsoft.com/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store>
- Books: "T-SQL Fundamentals" by Itzik Ben-Gan